



Database Administration and Management

Assignment 1

Submitted To

Sir Adnan Ashraf

Submitted By

Ahmad Shahzad

Roll No 22101003-093

Department Bachelor of Science in Information Technology Semester 8th

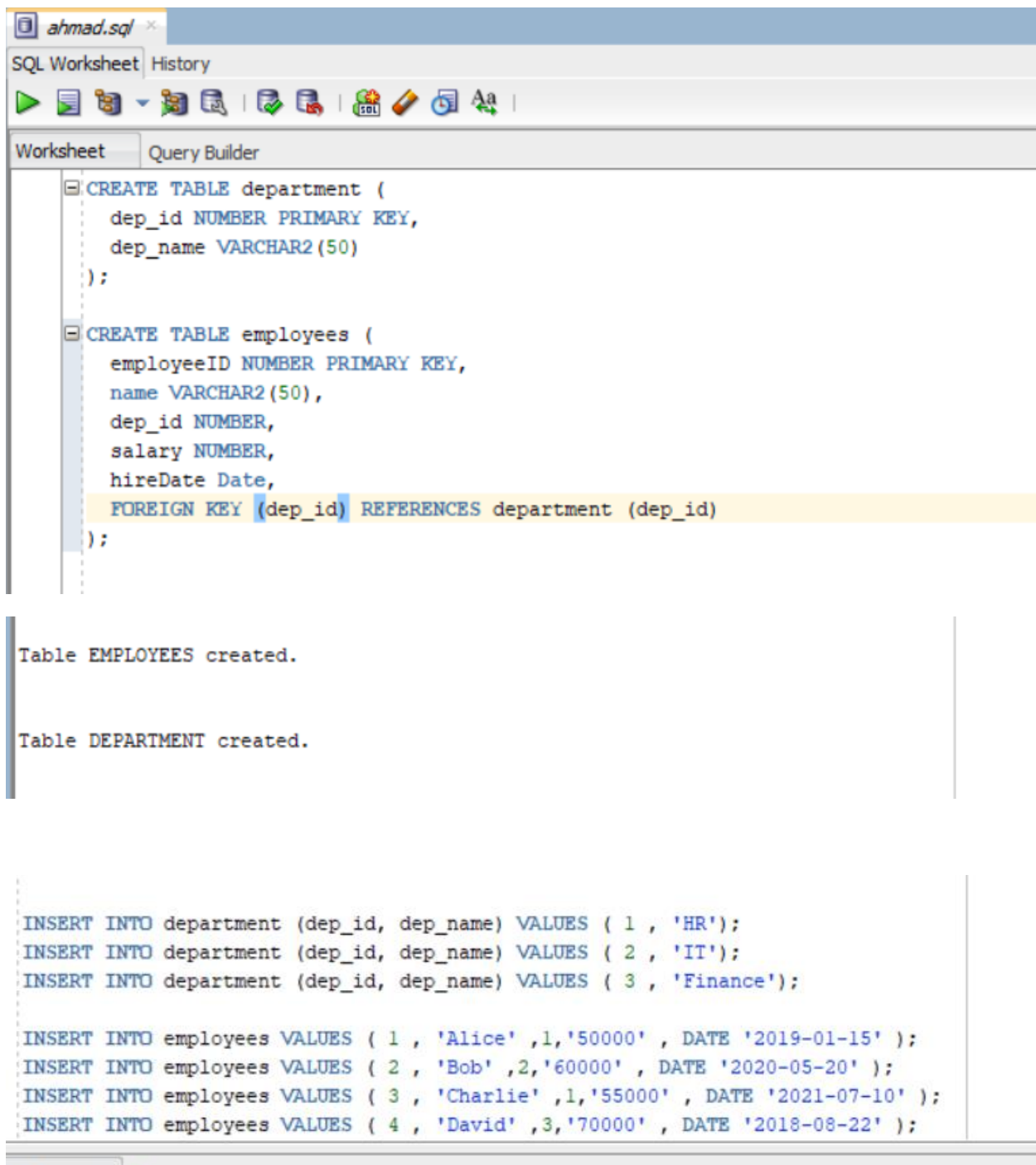
BS-IT

Session

2022-2026

Part 1: Basic SQL Queries

Creating tables and inserting data.



The screenshot shows an SQL IDE window titled 'ahmad.sql'. The interface includes a toolbar with icons for running queries, saving, and other database operations. The main editor area contains two SQL queries for creating tables and inserting data. The first query creates a 'department' table with 'dep_id' as the primary key and 'dep_name' as a VARCHAR2(50). The second query creates an 'employees' table with 'employeeID' as the primary key, 'name' as a VARCHAR2(50), 'dep_id' as a foreign key to the 'department' table, 'salary' as a NUMBER, and 'hireDate' as a DATE. Below the queries, the output area shows the successful creation of the 'EMPLOYEES' and 'DEPARTMENT' tables. At the bottom, a separate section displays four INSERT statements for the 'department' and 'employees' tables.

```
CREATE TABLE department (  
    dep_id NUMBER PRIMARY KEY,  
    dep_name VARCHAR2(50)  
);  
  
CREATE TABLE employees (  
    employeeID NUMBER PRIMARY KEY,  
    name VARCHAR2(50),  
    dep_id NUMBER,  
    salary NUMBER,  
    hireDate Date,  
    FOREIGN KEY (dep_id) REFERENCES department (dep_id)  
);  
  
Table EMPLOYEES created.  
  
Table DEPARTMENT created.  
  
INSERT INTO department (dep_id, dep_name) VALUES ( 1 , 'HR');  
INSERT INTO department (dep_id, dep_name) VALUES ( 2 , 'IT');  
INSERT INTO department (dep_id, dep_name) VALUES ( 3 , 'Finance');  
  
INSERT INTO employees VALUES ( 1 , 'Alice' ,1,'50000' , DATE '2019-01-15' );  
INSERT INTO employees VALUES ( 2 , 'Bob' ,2,'60000' , DATE '2020-05-20' );  
INSERT INTO employees VALUES ( 3 , 'Charlie' ,1,'55000' , DATE '2021-07-10' );  
INSERT INTO employees VALUES ( 4 , 'David' ,3,'70000' , DATE '2018-08-22' );
```

```
1 row inserted.
```

```
1 row inserted.
```

```
1 row inserted.
```

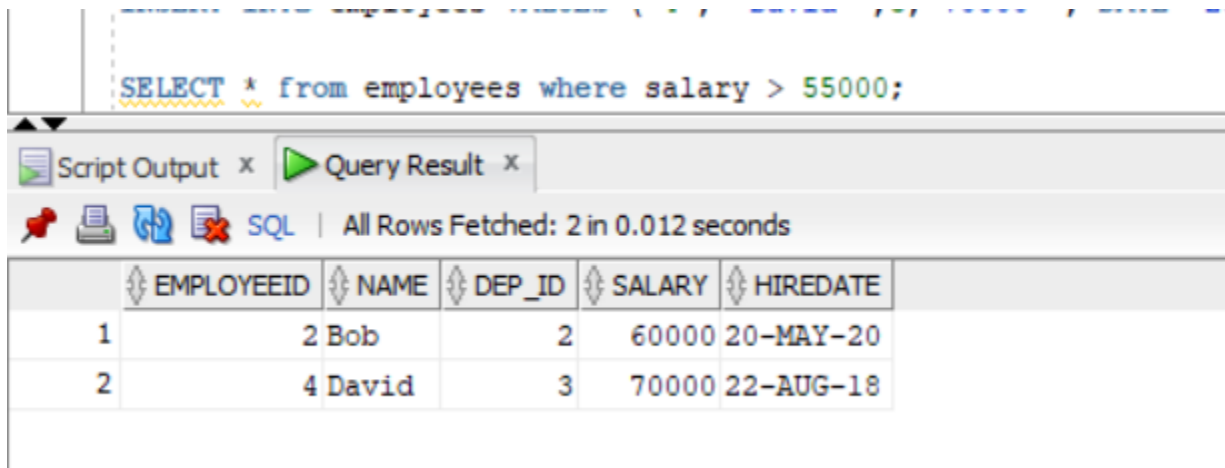
```
1 row inserted.
```

```
1 row inserted.
```

```
1 row inserted.
```

```
1 row inserted.
```

1. Write a query to display all employees with salaries above 55000.



The screenshot shows a SQL query execution interface. The query entered is `SELECT * from employees where salary > 55000;`. The results are displayed in a table with the following columns: EMPLOYEEID, NAME, DEP_ID, SALARY, and HIREDATE. The table contains two rows of data.

	EMPLOYEEID	NAME	DEP_ID	SALARY	HIREDATE
1	2	Bob	2	60000	20-MAY-20
2	4	David	3	70000	22-AUG-18

2. Retrieve employees' names and their hire dates, sorted by most recent hire.

```
select name, hireDate FROM employees ORDER BY hiredate DESC;
```

	NAME	HIREDATE
1	Charlie	10-JUL-21
2	Bob	20-MAY-20
3	Alice	15-JAN-19
4	David	22-AUG-18

- Find the total number of employees in each department using GROUP BY.

```
select dep_id , COUNT(*) AS total_employees FROM employees GROUP BY dep_id;
```

	DEP_ID	TOTAL_EMPLOYEES
1	1	2
2	2	1
3	3	1

Part 2: SQL Joins (INNER, LEFT, RIGHT, FULL, CROSS JOIN)

- Use INNER JOIN to display employees along with their department names.

```
select e.name, d.dep_name From employees e
INNER JOIN department d ON e.dep_id = d.dep_id;
```

	NAME	DEP_NAME
1	Alice	HR
2	Bob	IT
3	Charlie	HR
4	David	Finance

- Write a query using LEFT JOIN to show all departments, including those with no employees.

```
select d.dep_name, e.name FROM department d
LEFT JOIN employees e ON d.dep_id = e.dep_id;
```

	DEP_NAME	NAME
1	HR	Alice
2	IT	Bob
3	HR	Charlie
4	Finance	David

- Retrieve all employees using RIGHT JOIN, ensuring all departments appear even if they have no employees.

```
select e.name , d.dep_name from employees e
RIGHT JOIN department d ON e.dep_id = d.dep_id;
```

	NAME	DEP_NAME
1	Alice	HR
2	Bob	IT
3	Charlie	HR
4	David	Finance

7. . Use FULL JOIN to get all employees and departments, even if there is no matching data.

```
select e.name , d.dep_name from employees e |
FULL JOIN department d on e.dep_id = d.dep_id;
```

	NAME	DEP_NAME
1	Alice	HR
2	Bob	IT
3	Charlie	HR
4	David	Finance

Part 3: Advanced Joins and SQL Challenges

Additional Tables for Complex Joins

```
CREATE TABLE projects (  
    p_id NUMBER PRIMARY KEY,  
    p_name VARCHAR2(50),  
    dep_id NUMBER,  
    FOREIGN KEY (dep_id) REFERENCES department (dep_id)  
);  
  
CREATE TABLE employeeProjects (  
    employeeID NUMBER,  
    p_id NUMBER,  
    role VARCHAR2(20),  
    FOREIGN KEY (employeeID) REFERENCES employees(employeeID),  
    FOREIGN KEY (p_id) REFERENCES projects(p_id)  
);
```

Table PROJECTS created.

Table EMPLOYEEPROJECTS created.

Inserting Values

```

);

INSERT INTO projects VALUES (101, 'HR portal', 1);
INSERT INTO projects VALUES (102, 'IT security', 2);
INSERT INTO projects VALUES (103, 'Fianace app', 3);

INSERT INTO employeeProjects VALUES (1, 101, 'Manager');
INSERT INTO employeeProjects VALUES (2, 102, 'Developer');
INSERT INTO employeeProjects VALUES (3, 103, 'Aanlyst');
INSERT INTO employeeProjects VALUES (4, 101, 'HR Assistant');

```

Script Output x Query Result x

Task completed in 0.149 seconds

1 row inserted.

1 row inserted.

1 row inserted.

1 row inserted.

1 row inserted.

8. Write a query to list employees with their project names using JOINS.

```

select e.name, p.p_name FROM employees e
JOIN employeeProjects ep ON e.employeeID = ep.employeeID JOIN projects p ON ep.p_id = p.p_id;

```

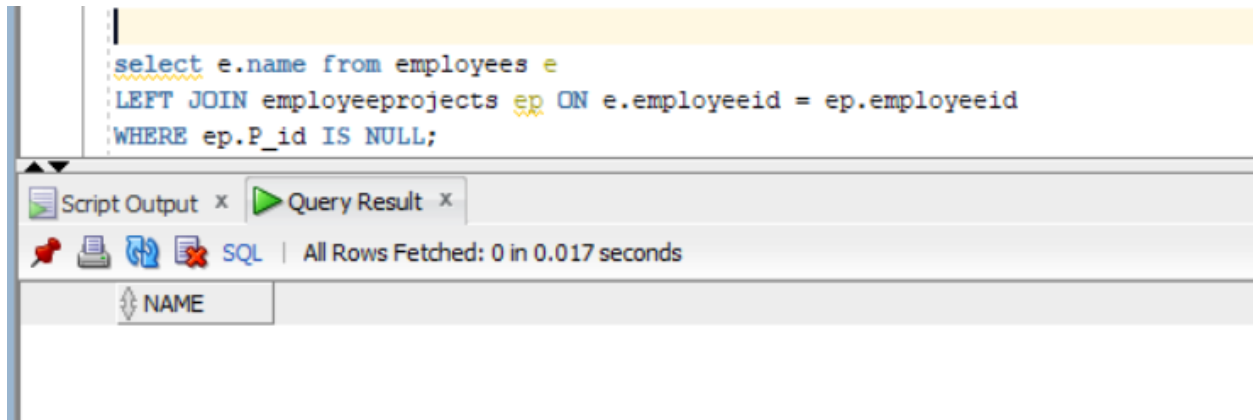
Script Output x Query Result x

SQL | All Rows Fetched: 4 in 0.011 seconds

	NAME	P_NAME
1	Alice	HR portal
2	Bob	IT Security
3	Charlie	Fianace
4	David	HR portal

9. Find employees who are not assigned to any project using LEFT JOIN and WHERE IS NULL.

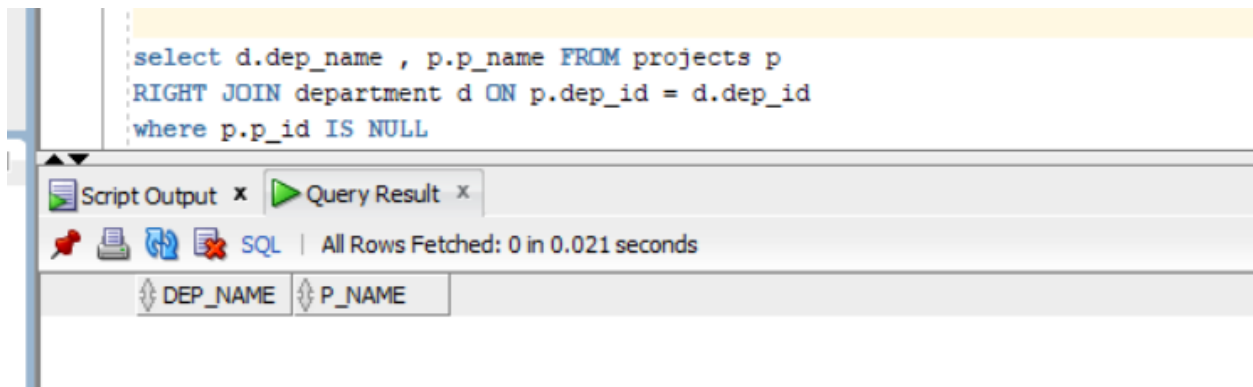
```
select e.name from employees e
LEFT JOIN employeeprojects ep ON e.employeeid = ep.employeeid
WHERE ep.P_id IS NULL;
```



The screenshot shows a SQL IDE interface. The top pane contains the SQL query: `select e.name from employees e LEFT JOIN employeeprojects ep ON e.employeeid = ep.employeeid WHERE ep.P_id IS NULL;`. The bottom pane has tabs for 'Script Output' and 'Query Result'. The 'Query Result' tab is active, showing a table with one column named 'NAME'. The status bar indicates 'All Rows Fetched: 0 in 0.017 seconds'.

10. Display departments with no projects assigned using a RIGHT JOIN

```
select d.dep_name , p.p_name FROM projects p
RIGHT JOIN department d ON p.dep_id = d.dep_id
where p.p_id IS NULL
```



The screenshot shows a SQL IDE interface. The top pane contains the SQL query: `select d.dep_name , p.p_name FROM projects p RIGHT JOIN department d ON p.dep_id = d.dep_id where p.p_id IS NULL`. The bottom pane has tabs for 'Script Output' and 'Query Result'. The 'Query Result' tab is active, showing a table with two columns: 'DEP_NAME' and 'P_NAME'. The status bar indicates 'All Rows Fetched: 0 in 0.021 seconds'.

11. Retrieve employees, their project roles, and department names in a single query

```

select e.name, ep.role, d.dep_name FROM employees e
JOIN employeeprojects ep ON e.employeeID = ep.employeeID
JOIN department d ON e.dep_id = d.dep_id;

```

	NAME	ROLE	DEP_NAME
1	Alice	Manager	HR
2	Charlie	Analyst	HR
3	Bob	Developer	IT
4	David	HR Assistant	Finance

12. List employees not assigned to any department, if any.

```

select name from employees where dep_id IS NULL;

```

NAME

13. Find employees who joined before 2020 and are working on at least one project

```

select e.name from employees e
JOIN employeeprojects ep ON e.employeeID = ep.employeeID
where e.hiredate < DATE '2020-01-01'

```

	NAME
1	Alice
2	David

14. Displaying all projects along with employee names (if assigned), ensuring projects with no employees are also shown.

```
select p.p_name , e.name from projects p
LEFT JOIN employeeprojects ep ON p.p_id = ep.p_id
LEFT JOIN employees e ON ep.employeeID = e.employeeID;
```

ipt Output x Query Result x

SQL | All Rows Fetched: 4 in 0.011 seconds

	P_NAME	NAME
1	HR portal	Alice
2	HR portal	David
3	IT Security	Bob
4	Fianace	Charlie

15. List departments and the number of projects assigned to each department

```
select d.dep_name, COUNT(p.p_id) AS total_projects FROM department d
LEFT JOIN projects p ON d.dep_id = p.dep_id
GROUP BY d.dep_id, d.dep_name;
```

Script Output x Query Result x

SQL | All Rows Fetched: 3 in 0.015 seconds

	DEP_NAME	TOTAL_PROJECTS
1	HR	1
2	Finance	1
3	IT	1